สรุปเนื้อหาและเฉลยคำถามท้ายบท สัปดาห์ที่ 9–14

จัดให้ครบทั้ง 6 บทจากเอกสารที่ส่งมา โดยแบ่งเป็น 2 ส่วนในแต่ละบท ได้แก่

1. สรุปเนื้อหาสำคัญ — อธิบายแนวคิดและคำศัพท์ที่ควรจำก่อนสอบ
2. เฉลยคำถามท้ายบท — ตอบครบทุกข้อ พร้อมคำอธิบายและตัวอย่างโค้ดในข้อที่ต้องเขียนโปรแกรม

คำตอบจะใช้ภาษาไทยแบบเข้าใจง่าย ระดับนิสิต สามารถนำไปอ่านทบทวนหรือปรับใช้เขียนส่งอาจารย์ได้

บทที่ 9: Unit & Integration Testing

8 คำถาม

ส่วนที่ 1 สรุปเนื้อหา

บทนี้เรียนเกี่ยวกับการทดสอบโปรแกรมอัตโนมัติ เพื่อให้มั่นใจว่าโปรแกรมทำงานถูกต้อง ลดข้อผิดพลาด และลดเวลาที่ต้องทดสอบด้วยมือ

1. ประเภทของ Testing

ประเภท	ความหมาย	ตัวอย่าง
Unit Testing	ทดสอบฟังก์ชันย่อยแยกเป็นส่วน ๆ	ทดสอบฟังก์ชันคำนวณราคา
Integration Testing	ทดสอบการทำงานร่วมกันของหลายส่วน	ทดสอบ API เชื่อมต่อฐานข้อมูล
End-to-End Testing	ทดสอบการทำงานตั้งแต่ต้นจนจบ	ทดสอบผู้ใช้ Login และเรียกดูข้อมูล

2. Jest

Jest เป็นเครื่องมือสำหรับเขียนและรันทดสอบ JavaScript

คำสั่งที่ควรรู้

- describe() ใช้จัดกลุ่มชุดทดสอบ
- test() หรือ it() ใช้เขียนกรณีทดสอบ
- expect() ใช้ตรวจสอบผลลัพธ์
- toBe() ตรวจสอบค่าที่คาดหวัง
- toEqual() ตรวจสอบค่าหรือโครงสร้าง Object
- beforeEach() ทำงานก่อนแต่ละ Test
- afterEach() ทำงานหลังแต่ละ Test

3. Arrange-Act-Assert (AAA)

เป็นรูปแบบการเขียน Test แบ่งเป็น 3 ขั้นตอน

1. Arrange – เตรียมข้อมูล
2. Act – เรียกใช้ฟังก์ชันที่ต้องการทดสอบ
3. Assert – ตรวจสอบผลลัพธ์

4. Supertest

ใช้ทดสอบ HTTP Endpoint ของ Express โดยไม่จำเป็นต้องเปิด Server และใช้พอร์ตจริง

5. Mocking

การจำลองการทำงานของสิ่งที่ระบบต้องพึ่งพา เช่น Database หรือ API ภายนอก เพื่อให้ทดสอบเฉพาะส่วนที่ต้องการได้ง่ายขึ้น

- jest.fn() จำลองฟังก์ชัน
- jest.mock() จำลอง Module

6. Test Coverage

ใช้ตรวจสอบว่าชุดทดสอบครอบคลุมโค้ดมากน้อยแค่ไหน โดยมี 3 ประเภทสำคัญ

- Statement Coverage – ตรวจสอบว่าคำสั่งถูกเรียกใช้งานหรือไม่
- Branch Coverage – ตรวจสอบว่าทุกเส้นทางเงื่อนไขถูกทดสอบหรือไม่
- Function Coverage – ตรวจสอบว่าฟังก์ชันถูกเรียกใช้งานหรือไม่

ข้อควรจำ: Coverage 100% ไม่ได้หมายความว่าโปรแกรมไม่มี Bug เพราะอาจเรียกใช้โค้ดครบ แต่ไม่ได้ตรวจสอบผลลัพธ์อย่างถูกต้อง

ส่วนที่ 2 เฉลยคำถามท้ายบท

ข้อ 1. จงอธิบายความแตกต่างระหว่าง Unit Testing, Integration Testing และ End-to-End Testing พร้อมยกตัวอย่างจากโปรเจกต์ student-api

คำตอบ:

- Unit Testing คือการทดสอบฟังก์ชันแต่ละส่วนแยกกัน เช่น ทดสอบฟังก์ชันตรวจสอบข้อมูลนิสิต
- Integration Testing คือการทดสอบการทำงานร่วมกัน เช่น ทดสอบ API ที่เชื่อมต่อกับ MySQL
- End-to-End Testing คือการทดสอบระบบตั้งแต่ต้นจนจบ เช่น ส่ง Request เพื่อเพิ่มข้อมูลนิสิต ตรวจสอบว่าข้อมูลถูกบันทึกลงฐานข้อมูล และสามารถเรียกดูข้อมูลได้

ข้อ 2. เพราะเหตุใดจึงต้องแยกไฟล์ app.js ออกจาก server.js ก่อนจึงจะใช้ Supertest ทดสอบ Express Endpoint ได้

คำตอบ:

เพราะ app.js ควรทำหน้าที่กำหนดค่า Express และ Routes ส่วน server.js ทำหน้าที่เปิด Server ด้วย app.listen()

การแยกไฟล์ทำให้สามารถนำ app ไปใช้กับ Supertest ได้โดยไม่ต้องเปิดพอร์ตจริง ลดปัญหาพอร์ตชนกัน และทำให้การทดสอบสะดวกขึ้น

ข้อ 3. จงอธิบายความหมายของโครงสร้าง Arrange-Act-Assert และยกตัวอย่างการเขียน Test

คำตอบ:

เป็นโครงสร้างการทดสอบโปรแกรม 3 ขั้นตอน ได้แก่ การเตรียมข้อมูล การเรียกใช้งาน และการตรวจสอบผลลัพธ์

ตัวอย่าง:

```
test('1 + 2 should equal 3', () => {
  // Arrange
  const a = 1;
  const b = 2;

  // Act
  const result = a + b;

  // Assert
  expect(result).toBe(3);
});
```

ข้อ 4. Mocking คืออะไร และเพราะเหตุใดการ Mock การเชื่อมต่อฐานข้อมูลจึงช่วยให้ Unit Test มีประสิทธิภาพมากขึ้น

คำตอบ:

Mocking คือการจำลองการทำงานของฟังก์ชันหรือ Module ที่ระบบต้องพึ่งพา

การ Mock ฐานข้อมูลช่วยให้ไม่ต้องเชื่อมต่อฐานข้อมูลจริง ทำให้ทดสอบได้รวดเร็ว ลดปัญหาจากข้อมูลจริง และสามารถกำหนดผลลัพธ์ที่ต้องการทดสอบได้

ข้อ 5. เพราะเหตุใด Test Coverage 100% จึงไม่ได้รับประกันว่าชุดทดสอบมีคุณภาพดี

คำตอบ:

เพราะ Coverage วัดเพียงว่าโค้ดถูกเรียกใช้งานมากน้อยแค่ไหน ไม่ได้วัดว่าการตรวจสอบผลลัพธ์ถูกต้องหรือไม่

ตัวอย่างเช่น โปรแกรมมีเงื่อนไขตรวจสอบอายุ แต่ Test เรียกใช้เฉพาะกรณีอายุมากกว่า 18 ปี โดยไม่ได้ตรวจสอบกรณีอายุต่ำกว่า 18 ปี แม้ Coverage บางประเภทจะสูง แต่ยังอาจมี Bug ได้

ข้อ 6. จงเขียน Test ด้วย Supertest สำหรับ Endpoint GET /api/v1/auth/me เพื่อตรวจสอบว่าเมื่อแนบ Token ที่ถูกต้อง ระบบคืน Status Code 200 พร้อมข้อมูลผู้ใช้

คำตอบ:

```
const request = require('supertest');

const app = require('../src/app');


describe('GET /api/v1/auth/me', () => {
  it('should return user information', async () => {
    const token = 'valid_test_token';


    const response = await request(app)
      .get('/api/v1/auth/me')
      .set('Authorization', `Bearer ${token}`);


    expect(response.status).toBe(200);
    expect(response.body).toHaveProperty('user');
  });
});
```

หมายเหตุ: ต้องใช้ Token ที่ระบบยอมรับจริง และปรับชื่อไฟล์กับรูปแบบ Response ให้ตรงกับโปรเจกต์

ข้อ 7. Statement Coverage, Branch Coverage และ Function Coverage แตกต่างกันอย่างไร

คำตอบ:

1. Statement Coverage ตรวจสอบว่าคำสั่งในโปรแกรมถูกเรียกใช้งานครบหรือไม่
2. Branch Coverage ตรวจสอบว่าทุกเส้นทางของเงื่อนไข เช่น True และ False ถูกทดสอบหรือไม่
3. Function Coverage ตรวจสอบว่าฟังก์ชันต่าง ๆ ถูกเรียกใช้งานครบหรือไม่

ข้อ 8. เพราะเหตุใดการทดสอบ Route ที่เชื่อมต่อฐานข้อมูลจริงซ้ำหลายครั้งโดยไม่มีการล้างข้อมูล อาจทำให้ Test ล้มเหลวในการรันครั้งถัดไป และมีวิธีแก้ไขอย่างไร

คำตอบ:

เพราะข้อมูลจากการทดสอบครั้งก่อนยังคงอยู่ในฐานข้อมูล เช่น การเพิ่มข้อมูลที่มี Email ซ้ำ ทำให้เกิดข้อผิดพลาดจาก Unique Constraint

วิธีแก้ไข ได้แก่

- ลบข้อมูลทดสอบหลังจากทดสอบเสร็จ
- ใช้ฐานข้อมูลสำหรับ Testing โดยเฉพาะ
- ใช้ beforeEach() เตรียมข้อมูลก่อนทดสอบ
- ใช้ afterEach() ล้างข้อมูลหลังทดสอบ

Pasted markdown.md

บทที่ 10: OpenAPI/Swagger & Developer Experience (DX)

8 คำถาม

ส่วนที่ 1 สรุปเนื้อหา

บทนี้เกี่ยวกับการทำเอกสาร API ให้เป็นมาตรฐาน เพื่อให้นักพัฒนาคนอื่นเข้าใจและสามารถนำ API ไปใช้งานได้ง่ายขึ้น

1. Developer Experience (DX)

คือประสบการณ์ของนักพัฒนาที่ใช้งาน API หากเอกสารไม่ชัดเจน จะทำให้เกิดความสับสน ใช้เวลาทำงานมากขึ้น และเกิดข้อผิดพลาดในการเชื่อมต่อระบบ

2. OpenAPI Specification

เป็นมาตรฐานสำหรับอธิบายรายละเอียด API โดยใช้ YAML หรือ JSON

องค์ประกอบหลัก ได้แก่

ส่วน	หน้าที่
openapi	ระบุเวอร์ชัน OpenAPI
info	รายละเอียด API
servers	URL ของ Server
paths	Endpoint ต่าง ๆ

components Schema และข้อมูลที่นำกลับมาใช้ซ้ำ

security รูปแบบการยืนยันตัวตน

3. Swagger UI

เครื่องมือแสดงเอกสาร OpenAPI ในรูปแบบหน้าเว็บไซต์ สามารถดูรายละเอียด Endpoint และทดลองส่ง Request ได้

4. Postman Collection

ใช้รวบรวม Request ของ API ไว้ในกลุ่มเดียวกัน สามารถแชร์และทดสอบซ้ำได้

5. Environment Variable

ใช้เก็บค่าที่เปลี่ยนแปลงตามสภาพแวดล้อม เช่น Base URL, Token หรือ Port ช่วยลดการแก้ไข URL ในทุก Request

6. Newman

เครื่องมือสำหรับรัน Postman Collection ผ่าน Command Line สามารถนำไปใช้ในกระบวนการทดสอบอัตโนมัติได้

7. API-First Design

เป็นแนวคิดที่ออกแบบ API และกำหนดสัญญาการใช้งานก่อนลงมือพัฒนา ทำให้ทีม Frontend และ Backend สามารถทำงานร่วมกันได้

ส่วนที่ 2 เฉลยคำถามท้ายบท

ข้อ 1. จงอธิบายความหมายของ Developer Experience (DX) และยกตัวอย่างผลกระทบที่เกิดขึ้นเมื่อ API มี DX ไม่ดี

คำตอบ:

DX คือประสบการณ์ของนักพัฒนาที่ต้องใช้งาน API ที่เราสร้างขึ้น

หาก DX ไม่ดี เช่น ไม่มีเอกสารบอก Endpoint หรือไม่ระบุรูปแบบข้อมูลที่ต้องส่ง จะทำให้นักพัฒนาเสียเวลาในการทำความเข้าใจ เกิดข้อผิดพลาด และทำงานร่วมกันได้ยาก

ข้อ 2. OpenAPI Specification ประกอบด้วยส่วนหลักใดบ้าง และแต่ละส่วนทำหน้าที่อะไร

คำตอบ:

1. openapi ระบุเวอร์ชันของมาตรฐาน OpenAPI
2. info ระบุชื่อ เวอร์ชัน และรายละเอียดของ API
3. servers ระบุ URL ของ Server
4. paths ระบุ Endpoint และ HTTP Method
5. components เก็บ Schema และส่วนประกอบที่ใช้ซ้ำ

6. security กำหนดรูปแบบ Authentication

ข้อ 3. เพราะเหตุใดการใช้ $ref อ้างอิง Schema ใน components.schemas จึงดีกว่าการเขียน Schema ซ้ำในทุก Endpoint

คำตอบ:

เพราะช่วยลดการเขียนโค้ดซ้ำ ทำให้เอกสารเป็นระเบียบ และหากต้องแก้ไขโครงสร้างข้อมูลก็สามารถแก้ไขที่ Schema กลางเพียงจุดเดียวได้

ข้อ 4. จงเขียนโครงสร้าง OpenAPI YAML สำหรับ Endpoint DELETE /api/v1/students/ ให้ครบทั้งกรณีสำเร็จ (200) และกรณีไม่พบข้อมูล (404)

คำตอบ:

```
openapi: 3.0.3


info:
  title: Student API
  version: 1.0.0


paths:
  /api/v1/students/{id}:
    delete:
      summary: Delete student by ID
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer


      responses:
        '200':
          description: Student deleted successfully
          content:
```

```
      application/json:
        schema:
          type: object
          properties:
            message:
              type: string
              example: Student deleted successfully

    '404':
      description: Student not found
      content:
        application/json:
          schema:
            type: object
            properties:
              message:
                type: string
                example: Student not found
```

ข้อ 5. Swagger UI ช่วยลดปัญหาเอกสารล้าสมัย (Stale Documentation) ได้อย่างไร

คำตอบ:

Swagger UI สามารถแสดงเอกสารจากไฟล์ OpenAPI ที่เชื่อมต่อกับแอปพลิเคชันโดยตรง เมื่อนักพัฒนาแก้ไขเอกสารและอัปเดตไฟล์ หน้า Swagger UI ก็สามารถแสดงข้อมูลใหม่ได้ ลดปัญหาเอกสารแยกจากโค้ดจริง

ข้อ 6. Postman Collection แตกต่างจากการทดสอบ Postman แบบเรียกทีละ Request อย่างไร และเหตุใดการใช้ Environment Variable จึงสำคัญ

คำตอบ:

Postman Collection เป็นการรวบรวม Request หลายรายการไว้ในกลุ่มเดียว สามารถจัดลำดับและทดสอบซ้ำได้ ส่วนการเรียกทีละ Request ต้องเลือกและส่งคำขอแยกกัน

Environment Variable ช่วยเก็บค่าที่เปลี่ยนแปลงได้ เช่น Base URL และ Token ทำให้เปลี่ยนสภาพแวดล้อมได้ง่ายโดยไม่ต้องแก้ไขทุก Request

ข้อ 7. Newman ทำหน้าที่คล้ายกับเครื่องมือใดที่เรียนในสัปดาห์ที่ 9 และแตกต่างกันอย่างไรในแง่ระดับของการทดสอบ

คำตอบ:

Newman ทำหน้าที่คล้าย Jest ในด้านการรันทดสอบอัตโนมัติ แต่แตกต่างกันดังนี้

- Jest ใช้ทดสอบในระดับ Code-level เช่น ฟังก์ชันและ Unit Test
- Newman ใช้รัน Postman Collection เพื่อทดสอบในระดับ HTTP-level เช่น Endpoint และ Response

ข้อ 8. จงอธิบายแนวคิด API-First Design และประโยชน์ต่อการทำงานร่วมกันระหว่างทีม Backend และ Frontend

คำตอบ:

API-First Design คือการออกแบบและกำหนดรายละเอียด API ก่อนเริ่มพัฒนาระบบจริง โดยใช้ OpenAPI เป็นสัญญากลางระหว่างทีม

ประโยชน์คือทีม Backend และ Frontend สามารถพัฒนางานคู่ขนานกันได้ โดยไม่ต้องรอให้ Backend พัฒนาเสร็จก่อน และช่วยลดปัญหาการส่งข้อมูลไม่ตรงกัน

Pasted markdown (2).md

บทที่ 11: External API Integration & Resilience

12 คำถาม

ส่วนที่ 1 สรุปเนื้อหา

บทนี้เรียนเกี่ยวกับการเชื่อมต่อ API ภายนอก และการออกแบบระบบให้สามารถทำงานต่อได้แม้บริการภายนอกเกิดปัญหา

1. External API

คือ API ที่ระบบของเราเรียกใช้งานจากผู้ให้บริการภายนอก เช่น การเรียก API เพื่อค้นหาข้อมูลหนังสือหรือข้อมูลสภาพอากาศ

ความเสี่ยงคือ API ภายนอกอาจล่ม ตอบสนองช้า หรือส่งข้อมูลผิดพลาด

2. Axios

เป็น Library สำหรับเรียก HTTP Request ใน Node.js โดยมีความสามารถ เช่น

- เรียก GET, POST, PUT และ DELETE
- แปลง JSON อัตโนมัติ
- กำหนด Timeout
- จัดการ Error ผ่าน try-catch

3. Retry Logic

คือการลองเรียก API ใหม่เมื่อเกิดความผิดพลาดชั่วคราว

ความผิดพลาดที่ควร Retry เช่น

- Network Error
- Timeout
- HTTP 500
- HTTP 503
- HTTP 429

ความผิดพลาดที่ไม่ควร Retry เช่น

- HTTP 400
- HTTP 401
- HTTP 404

4. Backoff

เป็นการเพิ่มระยะเวลารอระหว่างการ Retry

- Fixed Delay คือรอด้วยระยะเวลาเท่าเดิมทุกครั้ง
- Exponential Backoff คือเพิ่มเวลารอเป็นเท่าตัวในแต่ละครั้ง
- Jitter คือสุ่มเวลาเพิ่มเข้าไป เพื่อลดปัญหาที่ Request จำนวนมากกลับมาเรียกพร้อมกัน

5. Timeout

กำหนดเวลาสูงสุดที่ระบบจะรอ API ตอบกลับ เพื่อป้องกัน Request ค้างนานเกินไป

6. Fallback

คือวิธีสำรองเมื่อ API ภายนอกไม่สามารถใช้งานได้ เช่น

- ใช้ข้อมูลจาก Cache
- ใช้ข้อมูล Default
- แจ้งเตือนผู้ใช้ว่าบริการไม่พร้อมใช้งาน

7. Webhook

เป็นการสื่อสารที่บริการภายนอกส่งข้อมูลกลับมายังระบบของเราโดยอัตโนมัติ

แตกต่างจาก API ปกติที่ระบบของเราเป็นฝ่ายเริ่มเรียกบริการภายนอก

8. HMAC และ Signature

ใช้ตรวจสอบความถูกต้องและแหล่งที่มาของ Webhook โดยอาศัย Secret Key ร่วมกับ Hash Algorithm เช่น SHA-256

crypto.timingSafeEqual() ใช้เปรียบเทียบ Signature โดยลดความเสี่ยงจาก Timing Attack

ส่วนที่ 2 เฉลยคำถามท้ายบท

ข้อ 1. เพราะเหตุใดระบบที่เรียกใช้บริการภายนอกจึงต้องออกแบบให้ทนทานต่อความล้มเหลว (Resilience) มากกว่าระบบที่เชื่อมต่อเฉพาะฐานข้อมูลที่ตนเองควบคุม

คำตอบ:

เพราะระบบไม่สามารถควบคุมความพร้อมใช้งานของ API ภายนอกได้ หาก API ภายนอกล่มหรือช้าอาจส่งผลให้ระบบของเราทำงานผิดพลาดหรือค้างได้

จึงต้องมี Retry, Timeout และ Fallback เพื่อให้ระบบสามารถรับมือกับความผิดพลาดและให้บริการผู้ใช้ต่อได้

ข้อ 2. จงอธิบายความแตกต่างระหว่าง error.response ที่มีค่า กับกรณีที่ไม่มีค่า เมื่อเกิด Error จากการเรียก Axios

คำตอบ:

- error.response มีค่า หมายถึงได้รับ HTTP Response กลับมาจาก Server แล้ว แต่ Server ตอบกลับด้วย Status Code ที่ไม่สำเร็จ เช่น 404 หรือ 500
- error.response ไม่มีค่า หมายถึงไม่ได้รับ HTTP Response กลับมา เช่น Network Error หรือ Timeout

ข้อ 3. เพราะเหตุใด HTTP Status 400 Bad Request จึงไม่ควรถูก Retry ในขณะที่ 503 Service Unavailable ควรถูก Retry

คำตอบ:

HTTP 400 เกิดจาก Request ที่ส่งไปไม่ถูกต้อง การ Retry ด้วยข้อมูลเดิมจึงไม่สามารถแก้ปัญหาได้

ส่วน HTTP 503 หมายถึง Server ไม่พร้อมให้บริการชั่วคราว การ Retry ภายหลังอาจทำให้ระบบกลับมาใช้งานได้

ข้อ 4. จงเปรียบเทียบข้อดีข้อเสียระหว่าง Fixed Delay และ Exponential Backoff ในการออกแบบ Retry Logic

คำตอบ:

รูปแบบ	ข้อดี	ข้อเสีย
Fixed Delay	เข้าใจง่าย กำหนดเวลารอได้แน่นอน	อาจส่ง Request ซ้ำถี่เกินไป
Exponential Backoff	เพิ่มเวลารอ ช่วยลดภาระ Server	ต้องคำนวณเวลาและอาจรอนานขึ้น

ข้อ 5. เพราะเหตุใด Exponential Backoff แบบไม่มี Jitter จึงเสี่ยงต่อปัญหา Thundering Herd และ Jitter ช่วยแก้ปัญหานี้อย่างไร

คำตอบ:

เพราะ Request จำนวนมากที่เกิดปัญหาพร้อมกันจะมีเวลารอเท่ากัน และกลับมาเรียก Server พร้อมกันอีกครั้ง ทำให้ Server รับภาระหนัก

Jitter ช่วยโดยการสุ่มเวลารอของแต่ละ Request ให้แตกต่างกัน ทำให้ Request กระจายตัวและลดการเรียกพร้อมกัน

ข้อ 6. หากไม่กำหนด Timeout ให้กับการเรียก API ภายนอก จะเกิดผลกระทบใดต่อระบบ Backend ที่สร้างด้วย Node.js/Express

คำตอบ:

Request อาจค้างรอผลลัพธ์โดยไม่มีกำหนด ทำให้ผู้ใช้ต้องรอนาน และมีการใช้ทรัพยากร เช่น Connection และ Memory สำหรับ Request ที่ยังไม่เสร็จสิ้น ส่งผลให้ระบบรองรับผู้ใช้รายอื่นได้น้อยลง

ข้อ 7. จงยกตัวอย่างกลยุทธ์ Fallback อย่างน้อย 2 รูปแบบ พร้อมอธิบายสถานการณ์ที่เหมาะสมกับแต่ละรูปแบบ

คำตอบ:

1. Cache Fallback คือใช้ข้อมูลที่เคยบันทึกไว้ก่อนหน้า เหมาะกับระบบค้นหาข้อมูลที่ไม่จำเป็นต้องเป็นข้อมูลล่าสุดตลอดเวลา
2. Default Value Fallback คือใช้ข้อมูลสำรองที่กำหนดไว้ล่วงหน้า เหมาะกับระบบที่ต้องแสดงผลแม้ API ภายนอกจะล่ม
3. Error Message Fallback คือแจ้งผู้ใช้ว่าบริการไม่พร้อมใช้งาน เหมาะกับกรณีที่ไม่มีข้อมูลสำรองให้ใช้

ข้อ 8. Webhook แตกต่างจากการเรียก API ภายนอกแบบปกติ (Outbound) อย่างไร และเหตุใด Webhook Endpoint จึงควรตอบกลับด้วยความรวดเร็ว

คำตอบ:

API แบบ Outbound คือระบบของเราเป็นฝ่ายเรียก API ภายนอก ส่วน Webhook คือระบบภายนอกเป็นฝ่ายส่งข้อมูลเข้ามายังระบบของเรา

Webhook ควรตอบกลับอย่างรวดเร็ว เพราะหากตอบช้าอาจทำให้ผู้ให้บริการมองว่า Request ล้มเหลว และส่ง Event เดิมซ้ำอีกครั้ง

ข้อ 9. เพราะเหตุใด Webhook Endpoint ที่ไม่ตรวจสอบ Signature จึงเป็นช่องโหว่ตาม OWASP A08 และเพราะเหตุใดจึงต้องใช้ crypto.timingSafeEqual แทน === ในการเทียบ Signature

คำตอบ:

เพราะผู้โจมตีสามารถปลอม Request แล้วส่งข้อมูลเข้ามาในระบบได้ หากไม่มีการตรวจสอบ Signature ระบบจะไม่สามารถยืนยันได้ว่าข้อมูลมาจากผู้ให้บริการจริง

ส่วน crypto.timingSafeEqual() ช่วยลดความเสี่ยงจาก Timing Attack ซึ่งอาจเกิดจากระยะเวลาที่ใช้เปรียบเทียบข้อมูลแตกต่างกัน

ข้อ 10. HMAC คืออะไร และเพราะเหตุใดผู้โจมตีที่ไม่รู้ค่า Secret จึงไม่สามารถปลอม Signature ที่ถูกต้องได้ แม้จะรู้ชื่อ Algorithm ที่ใช้ เช่น SHA-256

คำตอบ:

HMAC คือวิธีสร้าง Message Authentication Code โดยใช้ Hash Algorithm ร่วมกับ Secret Key

ผู้โจมตีอาจรู้ว่าใช้ SHA-256 แต่หากไม่รู้ Secret Key ก็ไม่สามารถสร้าง Signature ที่ถูกต้องได้ง่าย ๆ เพราะต้องใช้ Secret เดียวกันในการคำนวณ

ข้อ 11. หากนำโค้ดตรวจสอบ Signature ในหัวข้อ 6.3 ที่ใช้ Hex ไปปรับใช้กับ Webhook ของ Shopify หรือ LINE โดยไม่แก้ไขอะไรเลย จะเกิดผลอย่างไร และต้องแก้โค้ดส่วนใด

คำตอบ:

การตรวจสอบอาจล้มเหลว เพราะผู้ให้บริการแต่ละรายอาจใช้รูปแบบการเข้ารหัส Signature และรูปแบบข้อมูลที่แตกต่างกัน

ต้องปรับส่วนที่เกี่ยวข้อง ได้แก่

- รูปแบบการแปลง Signature เช่น Hex หรือ Base64
- Algorithm ที่ใช้
- Header ที่ใช้รับ Signature
- วิธีสร้าง HMAC ให้ตรงกับเอกสารของผู้ให้บริการ

จึงต้องตรวจสอบเอกสารของ Shopify หรือ LINE ก่อนนำโค้ดไปใช้จริง

ข้อ 12. จงเขียนฟังก์ชัน Node.js ที่เรียก API ภายนอกด้วย Axios พร้อม Timeout 2 วินาที และ Retry สูงสุด 2 ครั้งเมื่อเกิด Network Error หรือ Status 5xx โดยใช้ Exponential Backoff พร้อม Jitter

คำตอบ:

```
const axios = require('axios');


async function fetchData(url) {
  const maxRetries = 2;


  for (let attempt = 0; attempt <= maxRetries; attempt++) {
```

```
    try {
      const response = await axios.get(url, {
        timeout: 2000
      });

      return response.data;

    } catch (error) {
      const isNetworkError = !error.response;
      const isServerError =
        error.response?.status >= 500;

      const shouldRetry =
        isNetworkError || isServerError;

      if (!shouldRetry || attempt === maxRetries) {
        throw error;
      }

      const baseDelay = 500 * (2 ** attempt);
      const jitter = Math.random() * 500;
      const delay = baseDelay + jitter;

      await new Promise(resolve =>
        setTimeout(resolve, delay)
      );
    }
  }
}
```

ฟังก์ชันนี้จะลองเรียก API สูงสุด 3 ครั้ง ได้แก่ ครั้งแรกและ Retry อีก 2 ครั้ง โดยแต่ละครั้งรอไม่เกิน 2 วินาที และเพิ่มเวลารอแบบ Exponential Backoff พร้อม Jitter

Pasted markdown (3).md

บทที่ 12: Advanced Input Validation & Rate Limiting

8 คำถาม

ส่วนที่ 1 สรุปเนื้อหา

บทนี้เกี่ยวกับการตรวจสอบข้อมูลที่ผู้ใช้ส่งเข้ามา และการจำกัดจำนวน Request เพื่อป้องกันการโจมตีระบบ

1. Input Validation

คือการตรวจสอบข้อมูลก่อนนำไปประมวลผล เช่น

- ตรวจสอบชนิดข้อมูล
- ตรวจสอบว่าข้อมูลจำเป็นครบหรือไม่
- ตรวจสอบความยาวของข้อความ
- ตรวจสอบรูปแบบข้อมูล
- ตรวจสอบความสัมพันธ์ระหว่าง Field

หากไม่มี Validation อาจทำให้เกิดช่องโหว่ เช่น Injection และข้อมูลผิดรูปแบบ

2. Zod

เป็น Library สำหรับตรวจสอบข้อมูลด้วย Schema

ตัวอย่าง:

```
const { z } = require('zod');


const userSchema = z.object({
  name: z.string().min(3),
  age: z.number().min(18),
  email: z.string().email()
});
```

ความหมายคือ

- name ต้องเป็น String และมีความยาวอย่างน้อย 3 ตัวอักษร

- age ต้องเป็น Number และมีค่าตั้งแต่ 18 ขึ้นไป
- email ต้องเป็นรูปแบบอีเมลที่ถูกต้อง

3. refine() และ superRefine()

- .refine() ใช้ตรวจสอบเงื่อนไขเพิ่มเติม โดยทั่วไปเหมาะกับกฎที่ต้องการตรวจสอบแล้วได้ผลผ่านหรือไม่ผ่าน
- .superRefine() ใช้ตรวจสอบเงื่อนไขที่ซับซ้อนกว่า และสามารถเพิ่ม Validation Error ได้หลายรายการ

4. Rate Limiting

คือการจำกัดจำนวน Request ที่ผู้ใช้สามารถส่งเข้ามาได้ภายในเวลาที่กำหนด เช่น Login ได้ไม่เกิน 5 ครั้งใน 1 นาที

ใช้ป้องกันการโจมตีแบบ Brute-force และลดการใช้งานระบบเกินความจำเป็น

5. Rate Limiting กับ Throttling

Rate Limiting	**Throttling**
จำกัดจำนวน Request	ควบคุมความเร็วของ Request
เมื่อเกินโควตาจะปฏิเสธ	อาจหน่วงเวลาการทำงาน
เหมาะกับป้องกัน Brute-force	เหมาะกับควบคุมอัตราการใช้งาน

6. express-rate-limit

เป็น Middleware สำหรับ Express ที่ใช้จำกัดจำนวน Request

ตัวอย่าง:

```
const rateLimit = require('express-rate-limit');


const limiter = rateLimit({
  windowMs: 60 * 1000,
  limit: 5,
  message: 'Too many requests'
});


app.use('/auth/login', limiter);
```

หมายถึงอนุญาตให้ส่ง Request ได้ 5 ครั้งภายใน 1 นาที

7. trust proxy

ใช้กำหนดให้ Express เชื่อถือข้อมูล IP ที่ส่งมาจาก Reverse Proxy เช่น Nginx หากตั้งค่าไม่ถูกต้องอาจทำให้ระบบตรวจสอบ IP ผิดพลาด

8. OWASP ZAP

เป็นเครื่องมือสแกนช่องโหว่ของ Web Application โดยอัตโนมัติ เช่น Security Headers ที่ขาดหายไป และปัญหา Injection บางประเภท

อย่างไรก็ตาม ผลการสแกนยังต้องให้ผู้พัฒนาตรวจสอบเพิ่มเติม

ส่วนที่ 2 เฉลยคำถามท้ายบท

ข้อ 1. จงอธิบายความแตกต่างระหว่าง .refine() และ .superRefine() ใน Zod พร้อมยกตัวอย่างสถานการณ์ที่ต้องใช้แต่ละแบบ

คำตอบ:

.refine() ใช้ตรวจสอบเงื่อนไขเพิ่มเติม โดยคืนค่า True หรือ False เช่น ตรวจสอบว่าวันสิ้นสุดต้องมากกว่าวันเริ่มต้น

ส่วน .superRefine() ใช้ตรวจสอบเงื่อนไขที่ซับซ้อนกว่า และสามารถเพิ่ม Error ได้หลายรายการในครั้งเดียว เช่น ตรวจสอบข้อมูลหลาย Field และแจ้งข้อผิดพลาดแยกตามแต่ละ Field

ข้อ 2. เพราะเหตุใด Rate Limiting เพียงอย่างเดียวจึงไม่สามารถทดแทน Input Validation ได้ และในทางกลับกัน

คำตอบ:

เพราะทั้งสองอย่างมีหน้าที่แตกต่างกัน

- Rate Limiting ป้องกันการส่ง Request จำนวนมากเกินกำหนด
- Input Validation ตรวจสอบความถูกต้องของข้อมูลที่ส่งเข้ามา

แม้ผู้ใช้จะส่ง Request เพียงครั้งเดียว แต่ข้อมูลอาจเป็นอันตรายหรือผิดรูปแบบได้ ดังนั้นจึงต้องใช้ทั้งสองอย่างร่วมกัน

ข้อ 3. จงเปรียบเทียบ Rate Limiting และ Throttling พร้อมยกตัวอย่างสถานการณ์ที่ควรเลือกใช้แต่ละแบบ

คำตอบ:

Rate Limiting เป็นการจำกัดจำนวน Request หากเกินกำหนดจะปฏิเสธ Request เช่น จำกัดการ Login ไม่เกิน 5 ครั้งต่อนาที เหมาะกับการป้องกัน Brute-force

Throttling เป็นการควบคุมความเร็วในการส่ง Request เช่น จำกัดการส่งข้อมูลให้ทำงานได้ตามอัตราที่กำหนด เหมาะกับระบบที่ต้องควบคุมปริมาณการใช้งานอย่างต่อเนื่อง

ข้อ 4. หากไม่ตั้งค่า trust proxy เมื่อระบบอยู่หลัง Reverse Proxy จะเกิดปัญหาใดกับ express-rate-limit

คำตอบ:

Express อาจมองเห็น IP ของ Reverse Proxy แทน IP จริงของผู้ใช้งาน ทำให้ผู้ใช้ทุกคนถูกนับรวมเป็น IP เดียวกัน และอาจถูกจำกัด Request ทั้งที่ไม่ได้ใช้งานเกินกำหนด

ข้อ 5. จงออกแบบ Zod Schema ที่ตรวจสอบว่า discountEndDate ต้องมาหลัง discountStartDate เสมอ โดยใช้ .refine()

คำตอบ:

```
const { z } = require('zod');


const discountSchema = z.object({
  discountStartDate: z.string().date(),
  discountEndDate: z.string().date()
}).refine((data) => {
  return new Date(data.discountEndDate) >
         new Date(data.discountStartDate);
}, {
  message: 'End date must be after start date',
  path: ['discountEndDate']
});
```

หลักการคือเปรียบเทียบวันที่สิ้นสุดกับวันที่เริ่มต้น หากวันที่สิ้นสุดไม่มากกว่าวันที่เริ่มต้น จะไม่ผ่าน Validation

ข้อ 6. เพราะเหตุใดการจำกัด Rate ตาม userId จึงแม่นยำกว่าการจำกัดตาม IP ในบาง Endpoint ยกตัวอย่างสถานการณ์ที่การจำกัดตาม IP ล้มเหลว

คำตอบ:

เพราะผู้ใช้หลายคนอาจใช้ IP เดียวกัน เช่น ผู้ใช้งานในมหาวิทยาลัยที่เชื่อมต่อผ่านเครือข่าย Wi-Fi เดียวกัน

หากจำกัดตาม IP ผู้ใช้ทุกคนอาจถูกนับรวมกัน แต่หากจำกัดตาม userId จะสามารถแยกโควตาของแต่ละบัญชีได้อย่างชัดเจน

ข้อ 7. OWASP ZAP ตรวจพบช่องโหว่ประเภทใดได้บ้าง และมีข้อจำกัดใดที่ทีมพัฒนายังต้องตรวจสอบด้วยตนเองเพิ่มเติม

คำตอบ:

OWASP ZAP สามารถตรวจพบปัญหาพื้นฐาน เช่น

- Missing Security Headers
- Missing Anti-CSRF Tokens
- SQL Injection บางกรณี
- การตั้งค่าความปลอดภัยที่ไม่เหมาะสม

ข้อจำกัดคือไม่สามารถรับประกันว่าจะตรวจพบช่องโหว่ทั้งหมดได้ ผู้พัฒนายังต้องตรวจสอบ Business Logic, Authentication และ Authorization ด้วยตนเอง

ข้อ 8. จงยกตัวอย่างมาตรการป้องกันจากสัปดาห์ที่ 4–12 อย่างน้อย 4 มาตรการ ที่ร่วมกันป้องกันหมวด A07 (Authentication Failures) ของ OWASP Top 10

คำตอบ:

1. JWT ใช้ตรวจสอบตัวตนของผู้ใช้งาน
2. RBAC ใช้กำหนดสิทธิ์การเข้าถึงตามบทบาท
3. Input Validation ใช้ตรวจสอบข้อมูลก่อนเข้าสู่ระบบ
4. Rate Limiting ใช้จำกัดจำนวนครั้งในการ Login เพื่อป้องกัน Brute-force
5. Helmet.js ช่วยกำหนด HTTP Security Headers

มาตรการเหล่านี้ทำงานร่วมกันเพื่อเพิ่มความปลอดภัยของระบบ Authentication

Pasted markdown (4).md

บทที่ 13: Microservices Architecture

8 คำถาม

ส่วนที่ 1 สรุปเนื้อหา

บทนี้เรียนเกี่ยวกับการออกแบบสถาปัตยกรรมระบบ โดยเปรียบเทียบ Monolithic และ Microservices รวมถึงวิธีแบ่งบริการและการสื่อสารระหว่างบริการ

1. Monolithic Architecture

เป็นสถาปัตยกรรมที่รวมส่วนต่าง ๆ ของระบบไว้ในแอปพลิเคชันเดียว เช่น ระบบจัดการหนังสือที่มีทั้งระบบผู้ใช้ ระบบหนังสือ และระบบยืมคืนอยู่ในโปรเจกต์เดียวกัน

ข้อดี:

- พัฒนาและเริ่มต้นได้ง่าย
- ทดสอบได้สะดวก

- จัดการ Transaction ได้ง่าย

ข้อเสีย:

- เมื่อระบบใหญ่ขึ้นอาจแก้ไขยาก
- หากส่วนสำคัญล้มเหลวอาจกระทบทั้งระบบ
- การขยายระบบต้องขยายทั้งแอปพลิเคชัน

2. Microservices Architecture

เป็นการแบ่งระบบออกเป็นบริการย่อย ๆ โดยแต่ละบริการมีหน้าที่รับผิดชอบชัดเจน และสามารถพัฒนา Deploy หรือ Scale แยกกันได้

ข้อดี:

- แต่ละบริการพัฒนาแยกกันได้
- ขยายเฉพาะบริการที่ต้องการได้
- ความผิดพลาดบางส่วนอาจไม่กระทบทั้งระบบ

ข้อเสีย:

- มีความซับซ้อนในการจัดการ
- ต้องจัดการการสื่อสารระหว่างบริการ
- ต้องดูแลความสอดคล้องของข้อมูล

3. Bounded Context

เป็นแนวคิดในการแบ่งบริการตามขอบเขตความรับผิดชอบทางธุรกิจ ไม่ใช่แบ่งตามชั้นทางเทคนิค

ตัวอย่างระบบจัดการหนังสือ:

- Book Service จัดการข้อมูลหนังสือ
- User Service จัดการข้อมูลสมาชิก
- Borrowing Service จัดการการยืมและคืนหนังสือ

4. API Gateway

เป็นจุดกลางที่รับ Request จาก Client และส่งต่อไปยังบริการที่เกี่ยวข้อง

หน้าที่หลัก ได้แก่

- Routing

- Authentication
- Rate Limiting
- Logging
- Response Aggregation

5. การสื่อสารระหว่างบริการ

มี 2 รูปแบบหลัก

รูปแบบ	**ความหมาย**	**ตัวอย่าง**
Synchronous	ส่ง Request และรอผลลัพธ์ทันที	ตรวจสอบข้อมูลสมาชิก
Asynchronous	ส่ง Event หรือ Message แล้วทำงานต่อโดยไม่ต้องรอผลลัพธ์	ส่งอีเมลแจ้งเตือน

6. Eventual Consistency

คือข้อมูลในแต่ละบริการอาจไม่ตรงกันทันที แต่จะสอดคล้องกันในที่สุดเมื่อทุกบริการประมวลผลเสร็จ

7. Circuit Breaker

เป็นรูปแบบการป้องกันระบบจากบริการที่ล้มเหลว โดยหยุดการเรียกบริการนั้นชั่วคราว เพื่อไม่ให้เกิด Request ซ้ำจนทำให้ระบบเสียหายมากขึ้น

8. Modular Monolith

คือการพัฒนาระบบในแอปพลิเคชันเดียว แต่แบ่ง Module ภายในให้มีขอบเขตหน้าที่ชัดเจน เป็นแนวทางที่เหมาะกับทีมขนาดเล็กก่อนพิจารณาแยกเป็น Microservices

ส่วนที่ 2 เฉลยคำถามท้ายบท

ข้อ 1. จงเปรียบเทียบข้อดีและข้อเสียของสถาปัตยกรรมแบบ Monolithic และ Microservices อย่างน้อยด้านละ 3 ประเด็น

คำตอบ:

หัวข้อ	**Monolithic**	**Microservices**
การพัฒนา	พัฒนาในระบบเดียว	พัฒนาแยกบริการ
การ Deploy	Deploy ทั้งระบบ	Deploy แยกบริการได้
การ Scale	ขยายทั้งระบบ	ขยายเฉพาะบริการ
การทดสอบ	ทดสอบง่ายกว่า	ซับซ้อนกว่า

การจัดการข้อมูล ใช้ฐานข้อมูลร่วมกันได้ง่าย ต้องจัดการข้อมูลหลายบริการ

ความซับซ้อน ต่ำกว่าในระบบขนาดเล็ก สูงกว่าในการดูแลระบบ

สรุป: Monolithic เหมาะกับระบบที่ไม่ซับซ้อน ส่วน Microservices เหมาะกับระบบขนาดใหญ่ที่ต้องการแยกการทำงานและขยายบริการอย่างอิสระ

ข้อ 2. เพราะเหตุใดการแบ่งบริการตามชั้นทางเทคนิค เช่น แยก Database Service และ Validation Service จึงมักนำไปสู่ปัญหา Distributed Monolith

คำตอบ:

เพราะบริการแต่ละส่วนยังต้องเรียกใช้งานกันตลอดเวลา และมีความสัมพันธ์กันอย่างใกล้ชิด ทำให้ไม่สามารถพัฒนาและ Deploy แยกจากกันได้อย่างอิสระ

จึงเกิดระบบที่แยกเป็นหลาย Service แต่ยังมีการผูกติดกันเหมือน Monolithic เดิม เรียกว่า Distributed Monolith

ข้อ 3. จงอธิบาย Bounded Context และยกตัวอย่างการแบ่งบริการจากระบบโครงงานของตนเองอย่างน้อย 3 บริการ

คำตอบ:

Bounded Context คือการแบ่งระบบตามขอบเขตความรับผิดชอบทางธุรกิจ โดยแต่ละบริการมีหน้าที่ชัดเจน

ตัวอย่างระบบจัดการหนังสือ (Book API):

1. Book Service รับผิดชอบเพิ่ม ลบ แก้ไข และค้นหาหนังสือ
2. Category Service รับผิดชอบจัดการหมวดหมู่หนังสือ
3. User Service รับผิดชอบจัดการข้อมูลผู้ใช้งาน

การแบ่งลักษณะนี้ช่วยให้แต่ละบริการมีหน้าที่ชัดเจนและสามารถพัฒนาแยกกันได้

ข้อ 4. API Gateway มีบทบาทอย่างไรในสถาปัตยกรรมแบบ Microservices และหากไม่มี Gateway จะเกิดปัญหาใดบ้าง

คำตอบ:

API Gateway ทำหน้าที่เป็นจุดกลางในการรับ Request จาก Client และส่งต่อไปยังบริการที่เกี่ยวข้อง รวมถึงจัดการ Authentication, Rate Limiting และ Logging

หากไม่มี Gateway Client ต้องเชื่อมต่อกับแต่ละ Service โดยตรง ทำให้ระบบซับซ้อน และต้องเขียนโค้ดจัดการการเชื่อมต่อซ้ำหลายจุด

ข้อ 5. จงเปรียบเทียบ Synchronous และ Asynchronous Communication ระหว่างบริการ พร้อมยกตัวอย่างสถานการณ์ที่เหมาะสมกับแต่ละรูปแบบ

คำตอบ:

- Synchronous คือการส่ง Request และรอ Response กลับมาทันที เหมาะกับการตรวจสอบข้อมูลที่ต้องใช้ผลลัพธ์ก่อนทำงานต่อ เช่น ตรวจสอบว่าหนังสือมีอยู่จริงหรือไม่
- Asynchronous คือการส่ง Message หรือ Event โดยไม่ต้องรอผลลัพธ์ทันที เหมาะกับงานเบื้องหลัง เช่น ส่งอีเมลแจ้งเตือนหลังจากมีการยืมหนังสือ

ข้อ 6. Circuit Breaker แตกต่างจาก Retry Logic ที่เรียนในสัปดาห์ที่ 11 อย่างไร และแก้ปัญหาใดที่ Retry Logic เพียงอย่างเดียวแก้ไม่ได้

คำตอบ:

Retry Logic คือการลองเรียกบริการใหม่เมื่อเกิดข้อผิดพลาดชั่วคราว

ส่วน Circuit Breaker คือการหยุดเรียกบริการที่มีปัญหาชั่วคราว เมื่อพบว่าบริการล้มเหลวต่อเนื่อง

Circuit Breaker ช่วยป้องกันไม่ให้ระบบส่ง Request ซ้ำไปยังบริการที่ยังไม่พร้อมใช้งาน ซึ่ง Retry เพียงอย่างเดียวอาจทำให้บริการนั้นรับภาระมากขึ้น

ข้อ 7. เพราะเหตุใดทีมขนาดเล็กจึงควรพิจารณาเริ่มต้นด้วย Modular Monolith ก่อนแยกเป็น Microservices แทนที่จะเริ่มต้นด้วย Microservices ตั้งแต่แรก

คำตอบ:

เพราะ Modular Monolith มีโครงสร้างที่จัดการง่ายกว่า ลดความซับซ้อนในการ Deploy การสื่อสารระหว่างบริการ และการดูแลฐานข้อมูล

ทีมขนาดเล็กจึงสามารถพัฒนาระบบให้มีขอบเขตชัดเจนก่อน แล้วค่อยแยกเป็น Microservices เมื่อมีความจำเป็นจริง

ข้อ 8. จงออกแบบการแบ่งบริการของระบบโครงงานของตนเองออกเป็นอย่างน้อย 3 บริการ พร้อมระบุว่าแต่ละบริการสื่อสารกันด้วยรูปแบบใด

คำตอบ:

ตัวอย่างระบบ Book API

Service	**หน้าที่**	**รูปแบบการสื่อสาร**
Book Service	จัดการข้อมูลหนังสือ	Synchronous
Category Service	จัดการหมวดหมู่	Synchronous
User Service	จัดการสมาชิก	Synchronous
Notification Service	ส่งการแจ้งเตือน	Asynchronous

ตัวอย่างการทำงาน:

1. ผู้ใช้ส่ง Request เพื่อเพิ่มหนังสือ
2. Book Service ติดต่อ Category Service เพื่อตรวจสอบหมวดหมู่ โดยใช้ Synchronous
3. เมื่อเพิ่มหนังสือสำเร็จ ระบบส่ง Event ไปยัง Notification Service
4. Notification Service รับ Event แล้วส่งการแจ้งเตือน โดยใช้ Asynchronous

การออกแบบนี้ช่วยแบ่งหน้าที่ของระบบออกจากกัน และทำให้แต่ละบริการสามารถรับผิดชอบงานของตนเองได้

Pasted markdown (5).md

บทที่ 14: Schema Evolution & Large-Scale Backend Design

8 คำถาม

ส่วนที่ 1 สรุปเนื้อหา

บทนี้เกี่ยวกับการเปลี่ยนแปลงโครงสร้างฐานข้อมูลอย่างปลอดภัย และการออกแบบระบบให้รองรับผู้ใช้งานจำนวนมาก

1. Schema Evolution

คือการเปลี่ยนแปลงโครงสร้างฐานข้อมูล เช่น การเพิ่ม Column การลบ Column หรือการเปลี่ยนชนิดข้อมูลในตาราง

หากแก้ไขฐานข้อมูลโดยไม่มีระบบ อาจทำให้ข้อมูลเดิมเสียหาย หรือระบบที่ใช้งานอยู่เกิดข้อผิดพลาด

2. Database Migration

เป็นกระบวนการจัดการการเปลี่ยนแปลงโครงสร้างฐานข้อมูลอย่างเป็นระบบ โดยใช้ Migration File

ฟังก์ชันสำคัญมี 2 ตัว

- up() ใช้ดำเนินการเปลี่ยนแปลงฐานข้อมูล
- down() ใช้ย้อนกลับการเปลี่ยนแปลง

ตัวอย่าง:

```
exports.up = function(knex) {
  return knex.schema.alterTable('students', table => {
    table.string('phone').nullable();
  });
};
```

```
exports.down = function(knex) {
  return knex.schema.alterTable('students', table => {
    table.dropColumn('phone');
  });
};
```

3. Nullable และ Default

เมื่อเพิ่ม Column ใหม่ในตารางที่มีข้อมูลอยู่แล้ว ควรกำหนดให้ Column สามารถรับค่า NULL หรือกำหนดค่า DEFAULT เพื่อไม่ให้เกิดปัญหากับข้อมูลเดิม

4. Expand-Contract Pattern

เป็นวิธีปรับเปลี่ยน Schema โดยไม่ทำให้ระบบหยุดทำงาน (Downtime)

มี 3 ขั้นตอนหลัก

1. Expand – เพิ่มโครงสร้างใหม่โดยยังคงของเดิมไว้
2. Migrate – ย้ายข้อมูลและปรับโค้ดให้ใช้โครงสร้างใหม่
3. Contract – ลบโครงสร้างเก่าหลังจากแน่ใจว่าไม่จำเป็นต้องใช้งานแล้ว

5. Scalability

คือความสามารถในการรองรับปริมาณผู้ใช้งานหรือ Request ที่เพิ่มขึ้น

มี 2 รูปแบบ

Vertical Scaling	**Horizontal Scaling**
เพิ่มทรัพยากรให้เครื่องเดิม	เพิ่มจำนวนเครื่องหรือ Instance
เพิ่ม CPU และ RAM	เพิ่ม Server
ตั้งค่าง่ายกว่า	ต้องจัดการหลาย Instance
มีข้อจำกัดด้าน Hardware	รองรับการขยายได้มากกว่า

6. Stateless Application

คือระบบที่ไม่เก็บสถานะการใช้งานของผู้ใช้ไว้เฉพาะใน Server เครื่องใดเครื่องหนึ่ง ทำให้ Request สามารถถูกส่งไปยัง Server เครื่องอื่นได้

เป็นสิ่งสำคัญสำหรับ Horizontal Scaling

7. Load Balancer

เป็นตัวกระจาย Request ไปยัง Server หลายเครื่อง เพื่อแบ่งภาระการทำงานและเพิ่มประสิทธิภาพระบบ

อัลกอริทึมสำคัญ ได้แก่

- Round Robin – กระจาย Request ตามลำดับ
- Least Connections – ส่งไปยัง Server ที่มีการเชื่อมต่อน้อยที่สุด
- IP Hash – ใช้ IP ของผู้ใช้ในการเลือก Server

8. Nginx

เป็น Web Server และ Reverse Proxy ที่สามารถนำมาใช้ทำ Load Balancing เพื่อกระจาย Request ไปยัง Backend หลาย Instance ได้

ส่วนที่ 2 เฉลยคำถามท้ายบท

ข้อ 1. เพราะเหตุใดการเพิ่ม Column แบบ NOT NULL โดยไม่มีค่า Default จึงเป็นอันตรายต่อตารางที่มีข้อมูลอยู่แล้ว

คำตอบ:

เพราะข้อมูลเดิมในตารางยังไม่มีค่าใน Column ใหม่ เมื่อเพิ่ม Column ที่บังคับให้ต้องมีค่าและไม่มี Default อาจทำให้ฐานข้อมูลไม่สามารถเพิ่ม Column ได้ หรือเกิดปัญหากับการ Insert ข้อมูลใหม่

จึงควรกำหนดให้รับค่า NULL ได้ หรือกำหนด Default Value เพื่อรองรับข้อมูลเดิม

ข้อ 2. จงอธิบายความแตกต่างระหว่างฟังก์ชัน up และ down ใน Migration File และยกตัวอย่างสถานการณ์ที่ต้องใช้ down

คำตอบ:

- up() ใช้เปลี่ยนแปลงโครงสร้างฐานข้อมูล เช่น เพิ่ม Column หรือตาราง
- down() ใช้ย้อนกลับการเปลี่ยนแปลง เช่น ลบ Column ที่เพิ่มเข้ามา

ตัวอย่างสถานการณ์ที่ต้องใช้ down() คือ เมื่อนักพัฒนาพบว่า Migration ที่เพิ่ม Column ใหม่ทำให้ระบบเกิดข้อผิดพลาด จึงต้องย้อนกลับไปใช้โครงสร้างเดิม

ข้อ 3. จงอธิบายขั้นตอนของ Expand-Contract Pattern พร้อมยกตัวอย่างการเปลี่ยนชื่อ Column โดยไม่มี Downtime

คำตอบ:

Expand-Contract Pattern เป็นวิธีเปลี่ยนแปลงโครงสร้างฐานข้อมูลโดยไม่ทำให้ระบบหยุดทำงาน

ตัวอย่างการเปลี่ยนชื่อ Column name เป็น full_name

1. Expand – เพิ่ม Column full_name เข้ามา โดยยังคง Column name ไว้

2. Migrate – คัดลอกข้อมูลจาก name ไปยัง full_name และปรับโค้ดให้ใช้ Column ใหม่
3. Contract – เมื่อตรวจสอบแล้วว่าระบบใช้ full_name ได้สมบูรณ์ จึงลบ Column name ออก

วิธีนี้ช่วยให้ระบบเดิมยังทำงานได้ระหว่างการเปลี่ยนแปลง

ข้อ 4. จงเปรียบเทียบ Vertical Scaling และ Horizontal Scaling อย่างน้อยด้านละ 3 ประเด็น

คำตอบ:

หัวข้อ	**Vertical Scaling**	**Horizontal Scaling**
วิธีเพิ่มประสิทธิภาพ	เพิ่ม CPU และ RAM	เพิ่มจำนวน Server
ความซับซ้อน	ตั้งค่าง่าย	ซับซ้อนกว่า
ค่าใช้จ่าย	อาจสูงเมื่อเพิ่ม Hardware	เพิ่มตามจำนวนเครื่อง
ข้อจำกัด	มีเพดาน Hardware	ขยายจำนวนเครื่องได้
การรองรับผู้ใช้	เพิ่มกำลังเครื่องเดิม	แบ่งภาระหลายเครื่อง

สรุป: Vertical Scaling เหมาะกับระบบที่ต้องการเพิ่มประสิทธิภาพเครื่องเดิม ส่วน Horizontal Scaling เหมาะกับระบบที่ต้องรองรับผู้ใช้งานจำนวนมาก

ข้อ 5. เพราะเหตุใดการออกแบบแอปพลิเคชันแบบ Stateless จึงเป็นเงื่อนไขจำเป็นสำหรับ Horizontal Scaling

คำตอบ:

เพราะ Stateless ทำให้ Server แต่ละเครื่องสามารถประมวลผล Request ได้โดยไม่ต้องพึ่งพาข้อมูลสถานะที่เก็บอยู่ในเครื่องใดเครื่องหนึ่ง

ดังนั้น Load Balancer จึงสามารถส่ง Request ไปยัง Server เครื่องใดก็ได้ ช่วยให้กระจายภาระงานและเพิ่มจำนวน Server ได้ง่ายขึ้น

ข้อ 6. จงเปรียบเทียบอัลกอริทึม Round Robin, Least Connections และ IP Hash พร้อมยกตัวอย่างสถานการณ์ที่เหมาะสมกับแต่ละแบบ

คำตอบ:

Algorithm	**การทำงาน**	**สถานการณ์ที่เหมาะสม**
Round Robin	กระจาย Request ตามลำดับ	Server มีประสิทธิภาพใกล้เคียงกัน
Least Connections	เลือก Server ที่มี Connection น้อยที่สุด	ระบบที่ Request ใช้เวลาทำงานต่างกัน

IP Hash ใช้ IP เพื่อเลือก Server ระบบที่ต้องการให้ผู้ใช้เดิมเข้าถึง Server เดิม

ข้อ 7. หาก Load Balancer ไม่ส่ง Header X-Forwarded-For จะส่งผลกระทบต่อ Rate Limiting ที่เรียนในสัปดาห์ที่ 12 อย่างไร

คำตอบ:

Express อาจไม่สามารถทราบ IP จริงของผู้ใช้งานได้ และมองเห็นเป็น IP ของ Load Balancer แทน

ทำให้ express-rate-limit นับ Request ของผู้ใช้ทุกคนรวมเป็น IP เดียวกัน ส่งผลให้ผู้ใช้ถูกจำกัด Request ผิดพลาด

ข้อ 8. จงเขียน Migration Script ที่เพิ่ม Column last_login_at ชนิด datetime ที่รับค่า NULL ได้ ในตาราง students พร้อม ฟังก์ชัน down ที่ถูกต้อง

คำตอบ:

```
exports.up = function(knex) {
  return knex.schema.alterTable('students', table => {
    table.dateTime('last_login_at').nullable();
  });
};

exports.down = function(knex) {
  return knex.schema.alterTable('students', table => {
    table.dropColumn('last_login_at');
  });
};
```

คำอธิบาย:

- up() เพิ่ม Column last_login_at
- dateTime() กำหนดชนิดข้อมูลเป็นวันและเวลา
- nullable() อนุญาตให้เก็บค่า NULL ได้
- down() ลบ Column ที่เพิ่มออก เพื่อย้อนกลับ Migration

Pasted markdown (6).md

สรุปภาพรวมทั้ง 6 บท สำหรับอ่านก่อนสอบ

ตารางนี้รวบรวมเนื้อหาที่ควรจำของแต่ละบทไว้ในที่เดียว

บท	หัวข้อ	สิ่งที่ต้องจำ
9	Testing	Jest, Supertest, Mocking, Coverage
10	API Documentation	DX, OpenAPI, Swagger, Postman, Newman
11	External API	Axios, Retry, Backoff, Timeout, Fallback, Webhook
12	Security	Zod, Validation, Rate Limiting, Throttling, ZAP
13	Architecture	Monolithic, Microservices, API Gateway, Circuit Breaker
14	Database & Scaling	Migration, up/down, Expand-Contract, Scaling, Load Balancer

คำศัพท์สำคัญที่มีโอกาสเจอในข้อสอบ

คำศัพท์	ความหมายสั้น ๆ
Unit Testing	ทดสอบหน่วยย่อย
Integration Testing	ทดสอบการทำงานร่วมกัน
E2E Testing	ทดสอบตั้งแต่ต้นจนจบ
Mocking	จำลองการทำงาน
Test Coverage	ความครอบคลุมของการทดสอบ
DX	ประสบการณ์ของนักพัฒนา
OpenAPI	มาตรฐานอธิบาย API
Swagger UI	หน้าแสดงเอกสาร API
Axios	Library สำหรับเรียก HTTP API
Retry	ลอง Request ใหม่
Timeout	จำกัดเวลารอ
Fallback	วิธีสำรองเมื่อระบบผิดพลาด
Webhook	การส่งข้อมูลกลับเข้าระบบ
HMAC	การตรวจสอบความถูกต้องด้วย Secret

Validation	ตรวจสอบความถูกต้องของข้อมูล
Rate Limiting	จำกัดจำนวน Request
Throttling	ควบคุมอัตราการส่ง Request
Microservices	แบ่งระบบเป็นบริการย่อย
API Gateway	จุดกลางรับและส่ง Request
Circuit Breaker	หยุดเรียกบริการที่ล้มเหลวชั่วคราว
Migration	จัดการเปลี่ยนแปลง Schema
Stateless	ไม่ผูกสถานะไว้กับ Server เครื่องใดเครื่องหนึ่ง
Load Balancer	กระจายภาระงาน
Horizontal Scaling	เพิ่มจำนวน Server
Vertical Scaling	เพิ่มทรัพยากรเครื่องเดิม

จุดที่แนะนำให้เน้นอ่านเป็นพิเศษ

บทที่ 9

- ความแตกต่างของ Testing ทั้ง 3 ประเภท
- Arrange-Act-Assert
- Mocking และ Test Coverage

บทที่ 10

- ส่วนประกอบ OpenAPI
- Swagger UI กับ Postman
- Newman และ API-First Design

บทที่ 11

- Retry กับ Timeout ต่างกันอย่างไร
- Fixed Delay กับ Exponential Backoff
- Jitter และ Fallback

- Webhook และ HMAC

บทที่ 12

- .refine() กับ .superRefine()
- Rate Limiting กับ Throttling
- trust proxy
- OWASP ZAP

บทที่ 13

- Monolithic กับ Microservices
- Synchronous กับ Asynchronous
- API Gateway และ Circuit Breaker

บทที่ 14

- Migration up() และ down()
- Expand-Contract Pattern
- Vertical กับ Horizontal Scaling
- Round Robin, Least Connections และ IP Hash

คำแนะนำในการอ่าน: อย่าจำแค่คำแปลภาษาอังกฤษ ควรจำให้ได้ว่าแต่ละเครื่องมือใช้แก้ปัญหาอะไร และแตกต่างจากเครื่องมือที่มีหน้าที่ใกล้เคียงกันอย่างไร เพราะคำถามท้ายบทหลายข้อเน้นการเปรียบเทียบและการยกตัวอย่างมากกว่าการท่องจำ

หมายเหตุ: เฉลยนี้ครอบคลุมคำถามท้ายบททั้งหมด 52 ข้อจากเอกสารสัปดาห์ที่ 9–14 โดยตัวอย่างโค้ดบางข้อเป็นตัวอย่างประกอบ ต้องปรับชื่อไฟล์และโครงสร้าง Response ให้ตรงกับระบบจริงก่อนนำไปใช้งาน